

Fan Director Studio — Database Schema (PostgreSQL)

Overview

Comprehensive relational database schema for Fan Director Studio. Uses PostgreSQL with UUIDs for all primary keys and timestamps in UTC.

Tables

1. users

Stores all user accounts (fans and creators).

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  name VARCHAR(255) NOT NULL,
  role ENUM('fan', 'creator', 'admin') NOT NULL DEFAULT 'fan',

  -- Creator-specific fields
  creator_handle VARCHAR(100) UNIQUE,
  creator_bio TEXT,
  creator_avatar_url VARCHAR(500),

  -- Fan profile
  fan_name VARCHAR(255),
  fan_avatar_url VARCHAR(500),

  -- Settings
  email_verified BOOLEAN DEFAULT FALSE,
  email_verified_at TIMESTAMP WITH TIME ZONE,
  is_active BOOLEAN DEFAULT TRUE,

  -- Metadata
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  last_login_at TIMESTAMP WITH TIME ZONE,

  CONSTRAINT valid_creator_handle CHECK (
    (role != 'creator' AND creator_handle IS NULL) OR
    (role = 'creator' AND creator_handle IS NOT NULL)
  )
);
```

```
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_creator_handle ON users(creator_handle) WHERE role = 'creator';
CREATE INDEX idx_users_role ON users(role);
```

Fields:

- `id`: Unique identifier (UUID)
- `email`: User's email address (unique)
- `password_hash`: Bcrypt hashed password
- `name`: Display name
- `role`: User type (fan, creator, admin)
- `creator_handle`: @username for creators (e.g., @maya_atelier)
- `creator_bio`: Creator's bio/description
- `creator_avatar_url`: Creator's profile image
- `fan_name`, `fan_avatar_url`: Optional fan profile details
- `email_verified`: Email verification status
- `is_active`: Account status
- `created_at`, `updated_at`: Timestamps
- `last_login_at`: Last login timestamp

2. `commission_requests`

Stores all fan commission requests.

```
CREATE TABLE commission_requests (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  creator_id UUID NOT NULL,
  fan_id UUID NOT NULL,

  -- Request details
  title VARCHAR(255) NOT NULL,
  description TEXT,

  -- Budget & pricing
  budget DECIMAL(10, 2) NOT NULL,
  estimated_total DECIMAL(10, 2),
  final_price DECIMAL(10, 2),

  -- Timeline
  requested_delivery_date DATE NOT NULL,
  estimated_delivery_date DATE,
  actual_delivery_date DATE,

  -- Status workflow
  status ENUM(
```

```

'draft',
'submitted',
'pending_creator_review',
'approved',
'changes_requested',
'in_production',
'delivered',
'declined'
) NOT NULL DEFAULT 'draft',

-- Reference & tracking
reference_image_url VARCHAR(500),
wardrobe_notes TEXT,
creator_boundaries TEXT,

-- Metadata
created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
submitted_at TIMESTAMP WITH TIME ZONE,
approved_at TIMESTAMP WITH TIME ZONE,
declined_at TIMESTAMP WITH TIME ZONE,

-- Soft delete
deleted_at TIMESTAMP WITH TIME ZONE,

FOREIGN KEY (creator_id) REFERENCES users(id) ON DELETE RESTRICT,
FOREIGN KEY (fan_id) REFERENCES users(id) ON DELETE CASCADE,

CONSTRAINT valid_budget CHECK (budget > 0),
CONSTRAINT valid_delivery_date CHECK (requested_delivery_date > CURRENT_DATE)
);

CREATE INDEX idx_requests_creator_id ON commission_requests(creator_id);
CREATE INDEX idx_requests_fan_id ON commission_requests(fan_id);
CREATE INDEX idx_requests_status ON commission_requests(status);
CREATE INDEX idx_requests_created_at ON commission_requests(created_at DESC);
CREATE INDEX idx_requests_creator_status ON commission_requests(creator_id, status);

```

Fields:

- `id` : Request UUID
- `creator_id` : Creator's user ID
- `fan_id` : Fan's user ID
- `title` : Commission title (e.g., "Vintage Lounge Greeting")
- `description` : Detailed request description
- `budget` : Fan's budget allocation
- `estimated_total` : Current scene card total estimate
- `final_price` : Final approved price (after creator review)
- `requested_delivery_date` : Fan's desired deadline
- `estimated_delivery_date` : Creator's estimate

- `actual_delivery_date` : When delivered
 - `status` : Request lifecycle status
 - `reference_image_url` : Reference image URL
 - `wardrobe_notes` : Creator's wardrobe approval notes
 - `creator_boundaries` : Creator's custom boundaries
 - `submitted_at`, `approved_at`, `declined_at` : Status transition timestamps
 - `deleted_at` : Soft delete timestamp
-

3. `scene_cards`

Stores scene card configurations for each request.

```
CREATE TABLE scene_cards (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  request_id UUID NOT NULL UNIQUE,

  -- Scene details
  title VARCHAR(255) NOT NULL,
  description TEXT,
  theme VARCHAR(100),
  mood VARCHAR(100),

  -- Components (JSON for flexibility)
  components JSONB NOT NULL DEFAULT '[]',
  line_items JSONB NOT NULL DEFAULT '[]',

  -- Pricing
  estimated_total DECIMAL(10, 2) NOT NULL DEFAULT 0,

  -- Media & references
  reference_image_url VARCHAR(500),
  video_duration_minutes INTEGER,

  -- Creator boundaries
  wardrobe VARCHAR(255),
  creator_approved_boundaries TEXT,

  -- Status
  is_locked BOOLEAN DEFAULT FALSE,
  locked_at TIMESTAMP WITH TIME ZONE,

  -- Metadata
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),

  FOREIGN KEY (request_id) REFERENCES commission_requests(id) ON DELETE CASCADE
);
```

```
CREATE INDEX idx_scene_cards_request_id ON scene_cards(request_id);
```

Fields:

- `id` : Scene card UUID
- `request_id` : Link to commission request (one-to-one)
- `title` : Scene card title (e.g., "Vintage Lounge Greeting")
- `description` : Scene description
- `theme` : Theme category (e.g., "vintage lounge")
- `mood` : Mood descriptor (e.g., "cozy evening")
- `components` : JSON array of scene components (duration, setup, greeting type)
- `line_items` : JSON array of pricing line items
- `estimated_total` : Sum of all line items
- `reference_image_url` : Reference photo URL
- `video_duration_minutes` : Video length
- `wardrobe` : Selected wardrobe option
- `creator_approved_boundaries` : Custom boundaries text
- `is_locked` : Whether scene card is finalized
- `locked_at` : When scene was locked

Sample Components JSON:

```
[
  {
    "id": "uuid",
    "name": "Duration",
    "value": "3-Minute Video",
    "icon": "lucide:clock",
    "price": 90,
    "isEditable": false
  },
  {
    "id": "uuid",
    "name": "Setting",
    "value": "Vintage Lounge Setup",
    "icon": "lucide:home",
    "price": 35,
    "isEditable": true
  }
]
```

4. `director_conversations`

Stores AI Director conversation history.

```
CREATE TABLE director_conversations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  request_id UUID NOT NULL,

  -- Message
  sender ENUM('fan', 'director') NOT NULL,
  content TEXT NOT NULL,

  -- Optional suggestions/choices
  suggestions JSONB DEFAULT '[]',

  -- Metadata
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),

  FOREIGN KEY (request_id) REFERENCES commission_requests(id) ON DELETE CASCADE
);

CREATE INDEX idx_director_conversations_request_id ON director_conversations(request_id);
CREATE INDEX idx_director_conversations_created_at ON director_conversations(created_at);
```

Fields:

- `id`: Message UUID
- `request_id`: Link to commission request
- `sender`: Who sent the message ('fan' or 'director')
- `content`: Message text
- `suggestions`: JSON array of suggested choices for user selection
- `created_at`: When message was created

5. creator_decisions

Tracks creator's approval decisions and responses.

```
CREATE TABLE creator_decisions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  request_id UUID NOT NULL,
  creator_id UUID NOT NULL,

  -- Decision type
  decision_type ENUM('approve', 'decline', 'request_changes', 'ask_question') NOT NULL,

  -- Decision details
  notes TEXT,
  changes_requested TEXT,
  revised_price DECIMAL(10, 2),
```

```

revised_delivery_date DATE,
decline_reason TEXT,

-- Question (for ask_question type)
question TEXT,
question_response TEXT,
question_answered_at TIMESTAMP WITH TIME ZONE,

-- Metadata
created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),

FOREIGN KEY (request_id) REFERENCES commission_requests(id) ON DELETE CASCADE,
FOREIGN KEY (creator_id) REFERENCES users(id) ON DELETE RESTRICT
);

CREATE INDEX idx_creator_decisions_request_id ON creator_decisions(request_id);
CREATE INDEX idx_creator_decisions_creator_id ON creator_decisions(creator_id);
CREATE INDEX idx_creator_decisions_type ON creator_decisions(decision_type);

```

Fields:

- `id` : Decision UUID
- `request_id` : Link to commission request
- `creator_id` : Creator's user ID
- `decision_type` : Type of decision (approve, decline, changes, question)
- `notes` : Optional approval notes
- `changes_requested` : What needs to change
- `revised_price` : New price if modified
- `revised_delivery_date` : New deadline if modified
- `decline_reason` : Why request was declined
- `question` : Clarifying question asked
- `question_response` : Fan's answer to question
- `question_answered_at` : When fan answered
- `created_at` , `updated_at` : Timestamps

6. `payments`

Tracks payment processing (for future integration).

```

CREATE TABLE payments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  request_id UUID NOT NULL UNIQUE,

  -- Payment details
  amount DECIMAL(10, 2) NOT NULL,

```

```

currency CHAR(3) NOT NULL DEFAULT 'USD',

-- Payment provider
provider ENUM('stripe', 'square', 'paypal') NOT NULL,
provider_payment_id VARCHAR(255) UNIQUE,

-- Status
status ENUM('pending', 'processing', 'completed', 'failed', 'refunded') NOT NULL DEFAULT 'pending',

-- Metadata
created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
completed_at TIMESTAMP WITH TIME ZONE,

-- Error tracking
error_message TEXT,
retry_count INTEGER DEFAULT 0,

FOREIGN KEY (request_id) REFERENCES commission_requests(id) ON DELETE RESTRICT,
CONSTRAINT valid_amount CHECK (amount > 0)
);

CREATE INDEX idx_payments_request_id ON payments(request_id);
CREATE INDEX idx_payments_status ON payments(status);
CREATE INDEX idx_payments_provider_payment_id ON payments(provider_payment_id);

```

Fields:

- `id`: Payment UUID
- `request_id`: Link to commission request
- `amount`: Payment amount
- `currency`: Currency code (e.g., 'USD')
- `provider`: Payment service (Stripe, Square, PayPal)
- `provider_payment_id`: External payment ID from provider
- `status`: Payment status
- `completed_at`: When payment completed
- `error_message`: Error details if failed
- `retry_count`: Number of retry attempts

7. `audit_logs`

Tracks all important actions for compliance and debugging.

```

CREATE TABLE audit_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID,

```

```

-- Action details
action VARCHAR(100) NOT NULL,
entity_type VARCHAR(50) NOT NULL,
entity_id UUID NOT NULL,

-- Changes (for updates)
changes JSONB,
old_values JSONB,
new_values JSONB,

-- Context
ip_address INET,
user_agent VARCHAR(500),

-- Metadata
created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),

FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
);

CREATE INDEX idx_audit_logs_user_id ON audit_logs(user_id);
CREATE INDEX idx_audit_logs_entity ON audit_logs(entity_type, entity_id);
CREATE INDEX idx_audit_logs_action ON audit_logs(action);
CREATE INDEX idx_audit_logs_created_at ON audit_logs(created_at DESC);

```

Fields:

- `id`: Log entry UUID
- `user_id`: User who performed action
- `action`: Action type (e.g., 'request_created', 'approved', 'declined')
- `entity_type`: What was affected (e.g., 'commission_request')
- `entity_id`: ID of affected entity
- `changes`: JSON of what changed
- `old_values`: Previous values
- `new_values`: New values
- `ip_address`: User's IP address
- `user_agent`: Browser user agent
- `created_at`: Timestamp

8. sessions

Manages user sessions (optional, for additional security).

```

CREATE TABLE sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,

```

```

-- Session token (hashed)
token_hash VARCHAR(255) UNIQUE NOT NULL,

-- Metadata
ip_address INET,
user_agent VARCHAR(500),

-- Expiry
expires_at TIMESTAMP WITH TIME ZONE NOT NULL,
created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
last_activity_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),

FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE INDEX idx_sessions_user_id ON sessions(user_id);
CREATE INDEX idx_sessions_expires_at ON sessions(expires_at);

```

Views (Optimized Queries)

Creator Dashboard View

```

CREATE VIEW creator_dashboard_view AS
SELECT
  cr.id,
  cr.title,
  cr.description,
  cr.estimated_total,
  cr.requested_delivery_date,
  cr.status,

  -- Fan info
  u.id as fan_id,
  u.name as fan_name,
  u.creator_handle as fan_handle,
  u.fan_avatar_url,

  -- Scene info
  sc.components,
  sc.line_items,

  -- Unresolved questions count
  COUNT(CASE WHEN cd.decision_type = 'ask_question' AND cd.question_answered_at IS NULL THEN 1 END) as unresolved_questions,

  cr.created_at
FROM commission_requests cr
JOIN users u ON cr.fan_id = u.id
LEFT JOIN scene_cards sc ON cr.id = sc.request_id

```

```
LEFT JOIN creator_decisions cd ON cr.id = cd.request_id
GROUP BY cr.id, u.id, sc.id;
```

Request Timeline View

```
CREATE VIEW request_timeline_view AS
SELECT
  cr.id,
  cr.fan_id,
  cr.creator_id,
  cr.submitted_at as submitted_date,
  cr.approved_at as approved_date,
  (SELECT created_at FROM payments WHERE request_id = cr.id AND status = 'completed' LIMIT 1) as payment_date,
  cr.estimated_delivery_date,
  cr.actual_delivery_date,
  cr.status
FROM commission_requests cr;
```

Indexes Summary

Table	Index	Purpose
users	email	Fast login lookups
users	creator_handle	Find creators by handle
commission_requests	creator_id, status	List creator's requests by status
commission_requests	fan_id	Find fan's requests
commission_requests	created_at	Sort by date
scene_cards	request_id	One-to-one lookup
director_conversations	request_id	Get conversation history
creator_decisions	request_id	Get decision history
creator_decisions	decision_type	Analytics queries
payments	status	Find pending payments
audit_logs	created_at	Chronological queries

Data Integrity Constraints

- 1. **Users:** Email unique, creator_handle unique (if creator)

2. **Commission Requests:** Budget > 0, delivery_date in future, one per fan per creator (optional)
3. **Payments:** Amount > 0, one per request, unique provider_payment_id
4. **Scene Cards:** One per request (unique constraint on request_id)
5. **Audit Logs:** Immutable (no updates after insert)
6. **Sessions:** Expires_at must be in future at creation

Migrations (Flyway/Alembic)

V1_initial_schema.sql

Contains all tables, indexes, and views above.

V2_add_audit_triggers.sql

```
CREATE FUNCTION audit_trigger()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO audit_logs (user_id, action, entity_type, entity_id, old_values, new_values)
    VALUES (
        CURRENT_SETTING('app.current_user_id', true)::uuid,
        TG_ARGV[0],
        TG_TABLE_NAME,
        CASE WHEN TG_OP = 'DELETE' THEN OLD.id ELSE NEW.id END,
        CASE WHEN TG_OP = 'UPDATE' THEN row_to_json(OLD) ELSE NULL END,
        CASE WHEN TG_OP = 'INSERT' OR TG_OP = 'UPDATE' THEN row_to_json(NEW) ELSE NULL END
    );
    RETURN CASE WHEN TG_OP = 'DELETE' THEN OLD ELSE NEW END;
END;
$$ LANGUAGE plpgsql;

-- Create triggers on important tables
CREATE TRIGGER commission_requests_audit
AFTER INSERT OR UPDATE OR DELETE ON commission_requests
FOR EACH ROW EXECUTE FUNCTION audit_trigger('commission_request');

CREATE TRIGGER creator_decisions_audit
AFTER INSERT OR UPDATE ON creator_decisions
FOR EACH ROW EXECUTE FUNCTION audit_trigger('creator_decision');
```

Backup Strategy

- **Daily backups:** Full backup at 2 AM UTC
- **Hourly incremental:** Every hour during business hours
- **Point-in-time recovery:** 30 days

- **Replication:** Standby replica for failover
 - **Testing:** Restore to staging monthly
-

Performance Considerations

1. Partition commission_requests by status (if table grows large)

```
CREATE TABLE commission_requests_submitted PARTITION OF commission_requests
FOR VALUES IN ('submitted', 'approved', 'in_production');
```

2. Archive old completed requests (yearly)

```
INSERT INTO commission_requests_archive
SELECT * FROM commission_requests
WHERE actual_delivery_date < NOW() - INTERVAL '1 year';
```

3. Denormalize heavily-read fields (e.g., estimated_total in requests table)

4. Connection pooling: Use PgBouncer for connection management

5. Query optimization: Use `EXPLAIN ANALYZE` for slow queries